

Introduction to Tries

Given a sequence of N words and a sequence of Q queries, design an algorithm to check if each of the Q queries are present in the given sequence of N words.

Example

Input:
 words = ["code", "coder", "coding", "block", "blocks", "news"]
 queries = ["code", "coding", "blocked", "block", "news", "new"]

Output:
 True, True, False, True, True, False

✓ ✓ ✗ ✓ ✓ ✗

1. using Linear Search

N words
 Q queries

1-query

In the worst-case we'll compare single query with all the N words.

$$N \cdot \min(\text{Wavg}, \text{Qavg})$$

$$Q \cdot N \cdot \min(\text{Wavg}, \text{Qavg})$$

time: $\therefore QN$

space: $O(1)$

2. using HashSet (unordered_set)

1. insert all the N words into a hash-set

2. iterate over all the queries and lookup each query in the hash-set

space:
 $O(N)$ due to hash-set

blocks
 111111
 blocked

time spent on a comparing two strs
 is $\propto \min(\text{len of } s_1, \text{len of } s_2)$

$N, Q \gg \text{Wavg}, \text{Qavg}$

$$97 \cdot 27^2 + 98 \cdot 27^1 + 99 \cdot 27^0 \rightarrow \text{hash-index}$$

(abc)₂₇
 $\downarrow$
 $27^2 \ 27^1 \ 27^0$

(1010)₂
 (197)₁₀

$$\Rightarrow N \cdot \frac{\text{Wavg}}{\text{const}} \uparrow \text{time for insertion}$$

$$Q \cdot \frac{\text{const}}{\sim \text{Qavg}}$$

3. using Tries

1. insert all the words in a trie $\rightarrow N \cdot \text{Wavg}$

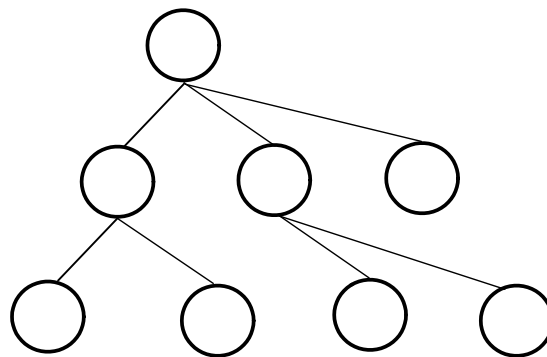
2. iterate over all the queries and lookup each query in the trie $\rightarrow Q \cdot \text{Qavg}$

$$N \cdot \text{Wavg} + Q \cdot \text{Qavg}$$

$$O(N + Q)$$

What are Tries ?

It is a data structure, specifically a kind of **generic tree** that supports efficient **lookups** and **insert** operations that on average take time proportional to the key length. A **generic tree** is a **tree** data-structure in which each node has **0** or **more** child nodes.

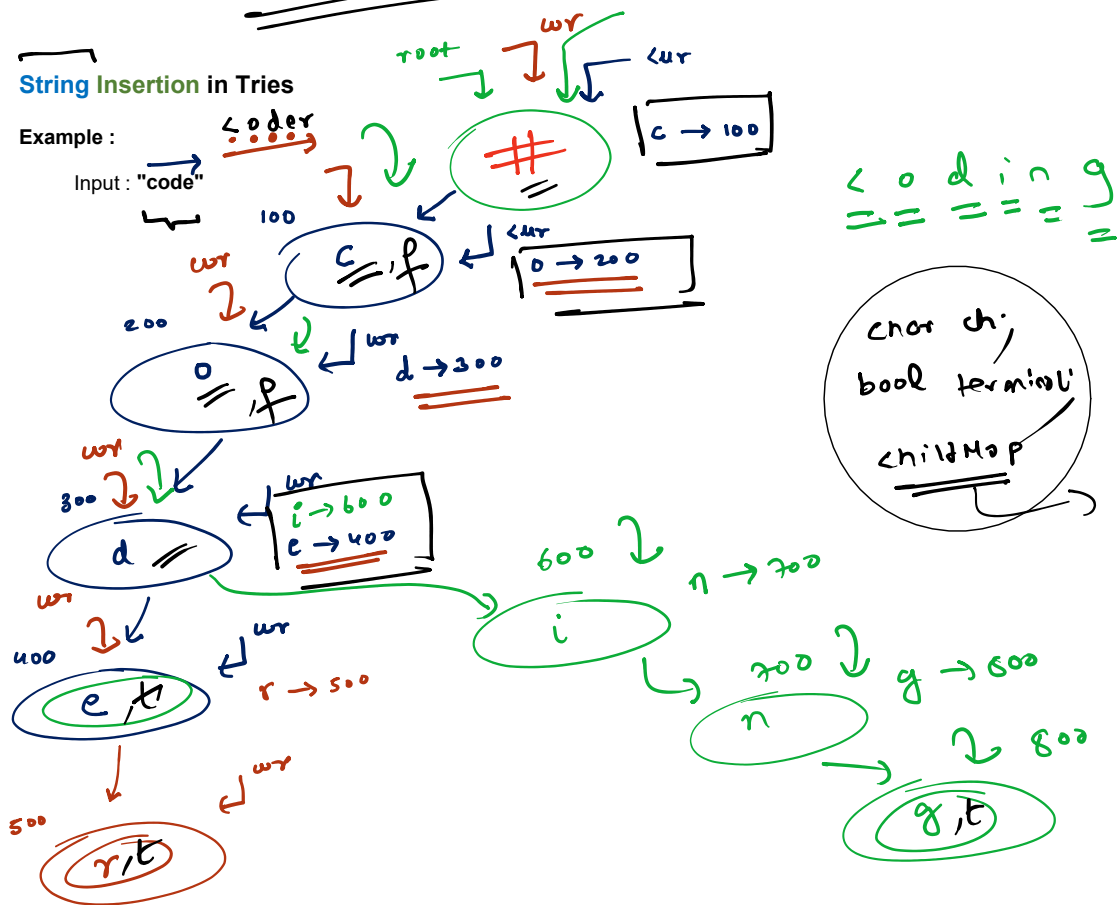


time $\propto$ len of word

String Insertion in Tries

Example :

Input : "code"

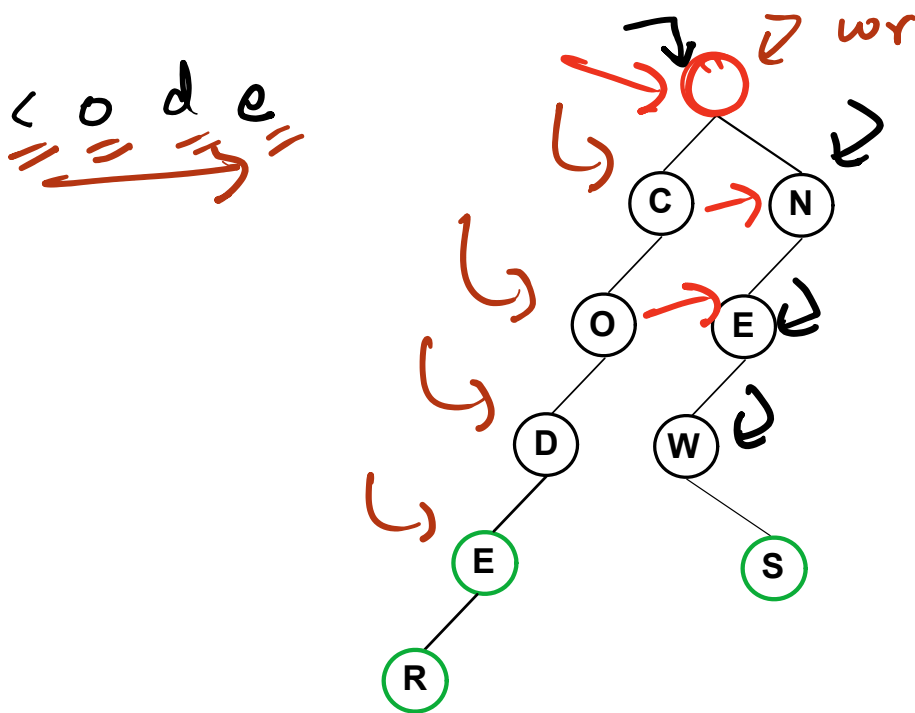


2021.9

char ch;
bool terminal;
childMap

unordered-map
 < char, node* >

String Lookup in Tries



new
===

neon
===

Shortest Unique Prefix Array

Given an array of strings, design an algorithm to find the **shortest unique prefix** for each string in the given array. If there is **no** unique prefix, return empty string.

Example

Input	code	coder	coding	new	neon
Output	""	"coder"	"codi"	"new"	"neo"

$\times c =$
 $\times co$
 $\times cod$
 $\times code$

==

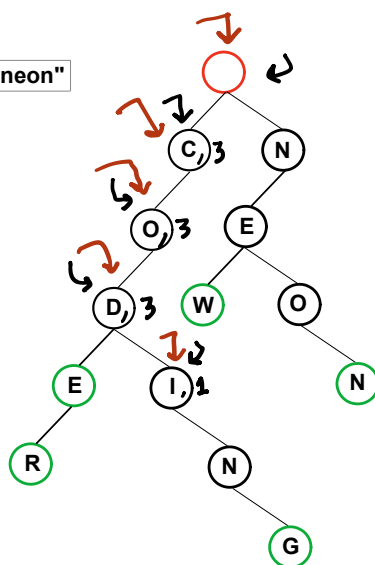
$\begin{matrix} c \\ co \\ cod \\ codi \\ coding \end{matrix}$

"code" "coder" "coding" "new" "neon"

==

"coding"

==

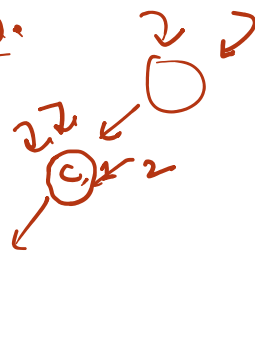


"coding"

ans = "codi"

coder

code



Longest Common Prefix

Given an array of strings, design an algorithm to **find** the **longest common prefix** string amongst the given strings. If there is **no** common prefix, return **empty** string.

Example

Input

"flow" "floor" "flys"

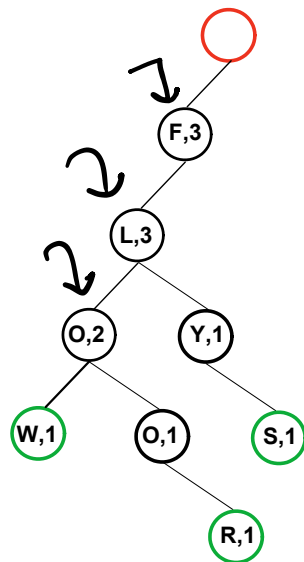
Output : "fl"

f "fl"

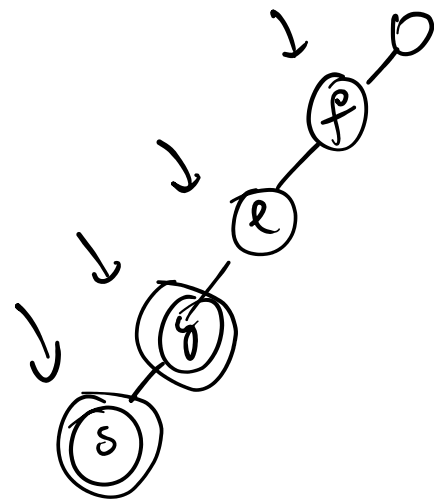
flow

flys

floor



ans = "fl"



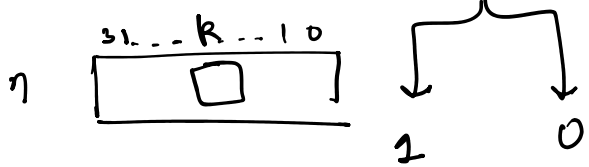
fly
flys

$$(27)_{10} \rightarrow (\quad)_2 \quad \begin{array}{r} 1 \quad 2 \quad 5 \quad 4 \quad \times \\ 4 \quad 3 \quad 2 \quad 1 \quad 0 \end{array}$$

$$\underline{\underline{(n \gg r) \& 1}}$$

$$27 = 16 + 8 + 2 + 1$$

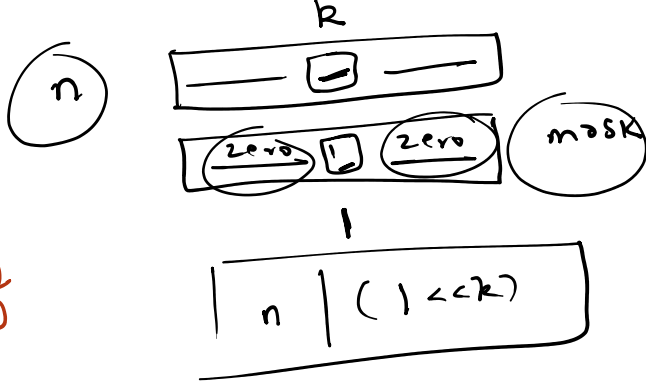
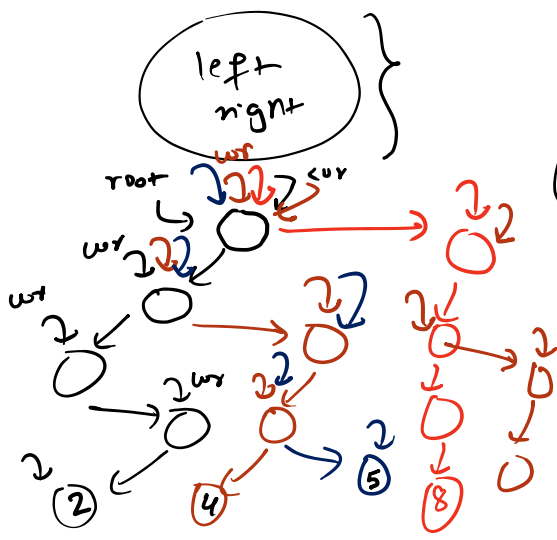
$$\begin{array}{cccc} \downarrow & \downarrow & \downarrow & \downarrow \\ 2^4 & 2^3 & 2^1 & 2^0 \end{array}$$



Integer **Insertion** in Tries

Example :

	4 bit	3rd	2nd	1st	0th
→	0	0	0	0	0
→	0	1	0	0	0
→	1	0	1	0	1
→	1	0	0	0	0
→	1	0	1	0	0

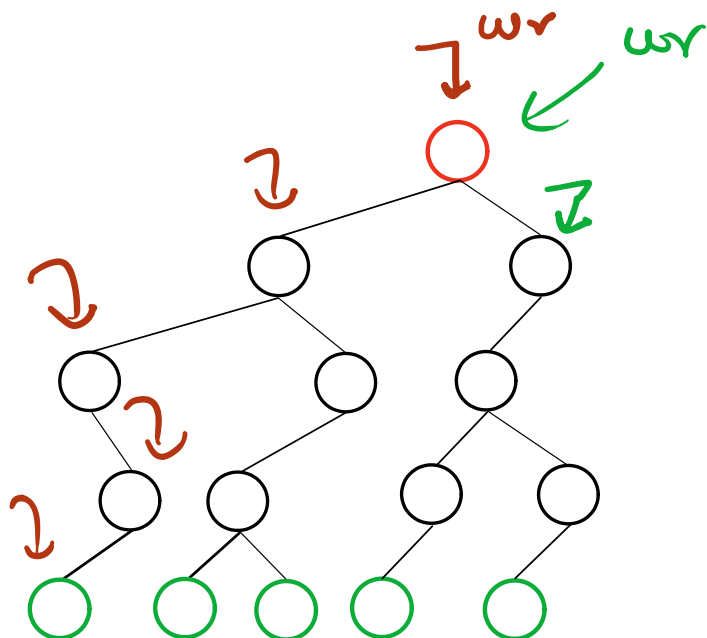


Integer Lookup in Tries

Example :

	3 rd	2 nd	1 st	0 th
2	0	0	1	0
4	0	1	0	0
5	0	1	0	1
8	1	0	0	0
10	1	0	1	0

1 1 0 0
==



Maximum XOR Pair

Given an array of **N** non-negative integers, design an algorithm to compute the **maximum XOR** possible between a **pair of integers** in the given array.

Example :

Input :

0	1	2	3	4
2	4	5	8	10

Output : 15

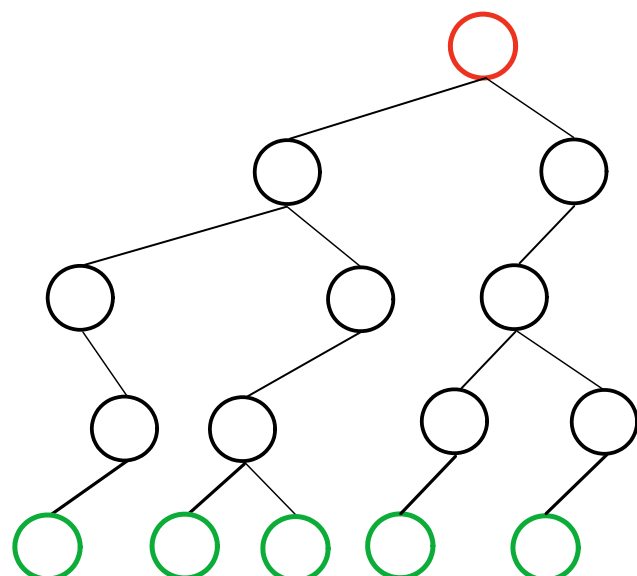
Explanation : $5^{10} = 15$

a	b	a^b
0	0	0
0	1	1
1	0	1
1	1	0

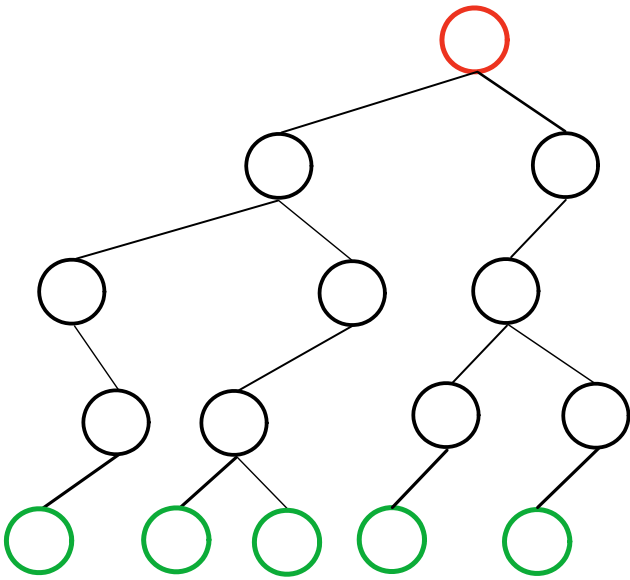
Input

0	1	2	3	4
2	4	5	8	10

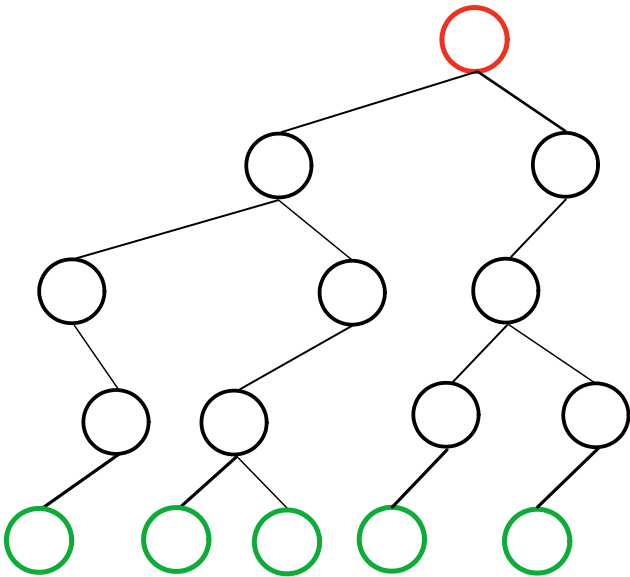
	3 rd	2 nd	1 st	0 th
2	0	0	1	0
4	0	1	0	0
5	0	1	0	1
8	1	0	0	0
10	1	0	1	0



	3 rd	2 nd	1 st	0 th
x = 2	0	0	1	0
y_{ideal}				
y_{actual}				
(x[^]y)_{ideal}	1	1	1	1
(x[^]y)_{actual}				



	3 rd	2 nd	1 st	0 th
x = 5	0	1	0	1
y_{ideal}				
y_{actual}				
(x[^]y)_{ideal}	1	1	1	1
(x[^]y)_{actual}				



Maximum Subarray XOR

Given an array of **N** integers, design an algorithm to find the **maximum subarray XOR**.

Example

Input

0	1	2	3	4	5
25	10	2	8	5	3

Output : 31

arr		0	1	2	3	4	5
		25	10	2	8	5	3
cxor	0	1	2	3	4	5	6
	0						

